CARE: Cascading Impact-Aware Compliance Test Suite Evolution under Regulatory Changes

ZHIYI XUE, East China Normal University, China
XIAOHONG CHEN*, East China Normal University, China
MIN ZHANG*, East China Normal University, China

In response to frequent changes in regulatory rules, this paper proposes CARE, a cascading impact-aware framework for automated compliance testing evolution. Existing approaches often suffer from over-reuse or missed updates because they treat rule changes in isolation and ignore complex inter-dependencies across testing artifacts. This paper highlights cascading impact propagation as a central challenge in regulation-driven test maintenance and shows that shifting from isolated rule handling to cascading impact-aware evolution is essential for achieving both high test quality and maintenance efficiency. Specifically, our CARE framework addresses this challenge by constructing a unified four-layer cascading relation model spanning Rule-Requirement-Scenario-Test Case, enabling fine-grained traceability across abstraction levels. By explicitly modeling how rule changes propagate and amplify along this chain, the framework precisely identifies impacted scenarios and test cases that need updating, while safely maximizing the reuse of unaffected ones. Experiments conducted on real-world compliance testing tasks across multiple domains show that CARE achieves an average F1 of 90.3% on updated test suites, outperforming existing methods by up to 164% and approaching expert-level effectiveness. Ablation studies further demonstrate that explicit cascading impact modeling and handling are key contributors to these improvements. In addition, CARE substantially reduces manual effort and improves test maintenance efficiency, and indicates strong cross-domain generalization.

CCS Concepts: • **Software and its engineering → Software testing and debugging**; **Maintaining software**; *Requirements analysis*.

Additional Key Words and Phrases: Compliance Testing; Test Suite Evolution; Change Impact Analysis; Requirements Traceability; Regulatory Compliance

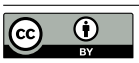
ACM Reference Format:
Zhiyi Xue, Xiaohong Chen, and Min Zhang. 2026. CARE: Cascading Impact-Aware Compliance Test Suite Evolution under Regulatory Changes. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA204 (October 2026), 24 pages. https://doi.org/10.1145/3832295

1 Introduction

In highly regulated industries such as finance, automotive, and power, software systems must undergo continuous compliance testing to ensure adherence to evolving regulations and standards [35, 36, 60, 61, 69, 73]. Regulatory authorities frequently revise existing rules or introduce new ones; for example, the NASDAQ exchange issued 112 rule revisions in 2025 alone [43]. Such frequent updates impose a substantial maintenance burden on software systems. Unlike internal

*Xiaohong Chen and Min Zhang are corresponding authors.

Authors' Contact Information: Zhiyi Xue, East China Normal University, Shanghai Key Laboratory of Trustworthy Computing, Shanghai, China, 52275902017@stu.ecnu.edu.cn; Xiaohong Chen, East China Normal University, Shanghai Key Laboratory of Trustworthy Computing, Shanghai, China, xhchen@sei.ecnu.edu.cn; Min Zhang, East China Normal University, Dishui Lake International Software Engineering Institute, Shanghai, China, zhangmin@sei.ecnu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA204
https://doi.org/10.1145/3832295

technical debt, which stems from design or implementation choices, compliance maintenance is externally driven by regulatory rule evolution. This evolution is inevitable and cumulative, steadily increasing the cost and complexity of maintaining compliance. As a result, test assets must be continuously updated to reflect rule changes, which significantly reduces system agility and imposes sustained maintenance pressure. Automating and optimizing regulatory test maintenance has therefore become a critical challenge for both research and industry.

In this context, automated test maintenance faces two fundamental challenges. First, the regulatory ecosystem is inherently fragmented: a system must simultaneously comply with rules issued by multiple independent authorities, each evolving at its own pace [16, 23, 43]. This results in a broad and fragmented maintenance scope. Second, and more critically, regulatory rules are rarely independent. They form intricate dependency networks in which changes to a small set of core rules can propagate through chains of related rules. Our empirical analysis of real-world rule evolution in the New York Stock Exchange (NYSE) and NASDAQ reveals that modifications to a few pivotal rules can indirectly affect far more test cases than those directly associated with the changed rules. These cascading impacts create a fundamental dilemma for test maintenance methods. If they reuse test cases solely based on direct rule-test links, they risk over-reuse by retaining test cases invalidated by indirect impacts, thereby introducing compliance risks. Conversely, when unable to precisely bound the propagation scope, they resort to coarse-grained updates, leading to insufficient updates and missed compliance violations. Although these two appear opposite, they stem from the same underlying deficiency: the lack of explicit modeling of cascading impacts.

Existing automated test reuse techniques have explored various directions, including test case feature analysis [3, 55, 58, 59], code similarity [33, 68], and requirements similarity [4, 34]. However, they exhibit inherent limitations in regulation-driven scenarios: (i) Abstraction-level mismatch. Approaches based on test case features or code similarity operate at a low abstraction level and cannot adequately capture semantic changes introduced at the regulatory rule level. (ii) Cascading impact blindness. Requirement-level similarity methods, though closer to our problem setting, treat requirements as isolated units and do not explicitly model inter-rule dependencies. Consequently, they detect only test cases directly linked to modified rules while overlooking indirect impacts propagated through dependency chains.

In summary, the core challenge of regulatory test maintenance is not merely detecting changed rules, but systematically modeling and controlling their cascading impacts across interdependent regulatory rules. Without explicit propagation modeling, maintenance remains unsafe: over-reuse retains invalid test cases, while missed updates overlook critical violations. Addressing cascading impacts is therefore not a minor enhancement to existing techniques, but a necessary paradigm shift from isolated rule-change handling to structured, impact-aware test suite evolution.

Motivated by this insight, we propose CARE, a **C**ascading Impact-**A**ware **R**egulatory Testing **E**volution framework for automated test suite evolution. The key idea is to explicitly model cross-level dependencies among regulatory artifacts and test assets, capturing both the direct and indirect effects of rule changes. At its core, CARE constructs a four-layer traceability model: Rule-Requirement-Scenario-Test Case. By first decomposing interdependent regulatory rules into independent requirements, then synthesizing related requirements into coherent scenarios, and finally anchoring historical test cases at the scenario level, CARE enables the structured aggregation of test cases into comprehensive test suites. More importantly, it explicitly models dependency chains among rules, allowing changes to be traced along propagation paths, including indirect and cascading impacts. This design enables CARE to precisely identify only the affected test suites, fully update them, and safely maximize the reuse of the unaffected suites. In this way, CARE systematically resolves the long-standing dilemma between over-reuse and missed updates, providing an effective solution for compliance maintenance.

We evaluate CARE on real-world compliance evolution tasks spanning stock trading, autonomous driving, and power control systems, covering diverse domains and highly heterogeneous regulatory frameworks. Across six financial datasets, CARE achieves an average F1 score of 90.3% on updated test suites, outperforming state-of-the-art (SOTA) reuse-based approaches, e.g., ESIM [4] and RPTSP [31], by up to 164%. Ablation studies further confirm that removing cascading impact modeling results in up to 84.4% performance degradation, demonstrating its critical role in capturing indirect rule dependencies. Beyond the financial domain, CARE consistently achieves F1 scores above 86.1% in automotive and power control systems, indicating strong cross-domain generalizability. These results not only establish cascading impact propagation as the central challenge in regulation-driven test suite evolution, but also validate that explicit impact-aware modeling is essential to ensure test quality, safe reuse, and efficient maintenance in real-world regulatory environments.

In summary, this paper makes the following contributions:

(1) We propose CARE, a cascading impact-aware test suite evolution framework that enables systematic and precise maintenance of test assets in dynamic regulatory environments by explicitly modeling and tracking rule change propagation.
(2) We design a four-layer traceability model that, for the first time, establishes fine-grained associations among regulatory rules, requirements, scenarios, and test cases, providing critical support for automated change impact analysis and precise reuse.
(3) We conduct extensive experiments on real-world compliance evolution tasks across multiple domains, demonstrating that the method maintains expert-level effectiveness (F1 > 86%) while significantly reducing maintenance costs and exhibiting excellent cross-domain generalization.

2 Related Work

In this section, we review related work on test case reuse, the construction of reusable test assets, and the analysis of requirement dependencies and change impact.

2.1 Test Case Reuse

Many studies have explored test case reuse based on either test case characteristics or code. Approaches based on test case characteristics extract reusable templates via structural abstraction [1, 2] or leverage similarity metrics such as clustering [58], test strategies and steps [59], and semantic function matching [29]. Code-based approaches achieve reuse by comparing code paths [68], execution traces [33], or function and variable similarities [39]. While effective in their respective settings, these methods operate at a low level of abstraction, which makes them insensitive to high-level regulatory rule changes and limits their applicability in compliance testing.

The work most closely related to ours focuses on requirements-based reuse, where test cases are identified according to similarities in requirement documents, functions, or products. For example, Chen et al. transformed functional descriptions into vectors and applied ESIM for text matching to detect similar requirements [4], while Ilays et al. proposed RPTSP, which uses semantic analysis and K-means to identify requirement changes and prioritize test case reuse [31]. However, both approaches consider functional points in isolation, ignoring interdependencies among requirements. Kang et al. targeted product lines [34], extracting domain test requirements and variation points to derive product-specific test cases from platform cases; yet this method relies on predefined static templates and struggles to handle dynamic requirement dependencies and cascading impacts.

2.2 Reusable Test Asset Construction

Many studies have built test asset repositories by integrating product information, requirements, test steps, test cases, and execution results to facilitate reuse. For instance, Devos et al. developed QPattern [9], a domain-specific repository for smart card applications that provides reusable

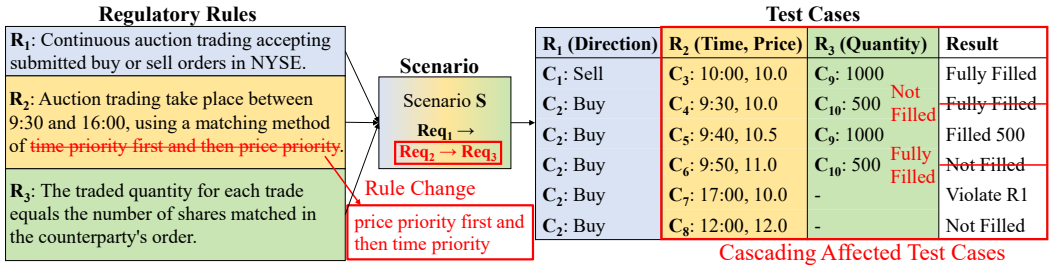


Fig. 1. An example of the cascading impact, where changing Rule R_2 propagates through Scenario S and affects Rule R_3 that depends on it, thereby invalidating their Test Cases C_3-C_{10}.

templates derived from structured test strategies; Li et al. proposed a five-component reuse model extracting documents, test terms, and test cases from repositories [40]; Patel et al. designed Harmony to organize test cases by products, modules, and business scenarios [56]; and Nan et al. extracted patterns from historical warship software test cases and applied semantic retrieval [46]. Some works leverage ontologies or knowledge graphs to enhance reuse. Cai et al. annotated test cases with functionality-based ontologies to enable semantic matching [3], while Yang et al. built a knowledge graph connecting components, products, and defects to recommend relevant test cases [71].

Overall, test asset repositories capture valuable historical test information for reuse. However, existing approaches do not establish direct traceability from regulatory rules to test cases and rely mainly on manual extraction and maintenance, which is labor-intensive and time-consuming.

2.3 Requirement Dependencies and Change Impact Analysis

Early studies refined dependency models to capture intrinsic and additional dependencies [72], analyzed large-scale change requests to reveal cascading-like propagation [27], or used fuzzy logic to classify strong and weak dependencies for regression test prioritization [10]. While insightful, these methods operate primarily at the requirements level and rely on manually established requirement-test links, leaving subsequent assets, such as test cases, outside automated analysis.

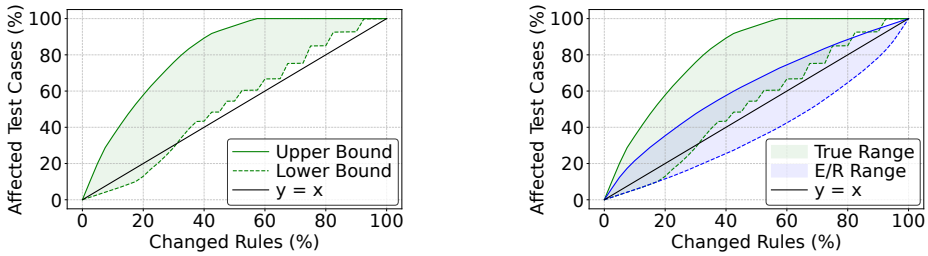
Broader research on requirements traceability and dependency extraction has employed information retrieval, machine learning, and dependency parsing to derive requirement links [30], with applications in impact analysis [64] and in linking regulatory to system-level requirements for security testing [65]. Nevertheless, automated test case update and reuse remain largely unaddressed.

3 Motivation: The Cascading Impact of Regulatory Changes

This section motivates the necessity of cascading impact awareness in test case update and reuse by examining how regulatory changes propagate from rules to requirements, then to scenarios, and ultimately to test cases. We argue that regulatory changes are rarely isolated: changes to a small subset of rules often affect interdependent requirements within a scenario, producing a *cascading impact* that invalidates a substantially larger set of test cases. We first illustrate this phenomenon with a concrete, intuitive example, then empirically demonstrate its presence in real-world regulatory rules, reveal that existing methods largely overlook it, and finally derive key design implications for effective, cascading impact-aware test case update and reuse.

3.1 Cascading Impacts in Regulatory Changes: Example and Empirical Evidence

We first illustrate how regulatory changes propagate through scenarios to test cases using a concrete example from the NYSE 2025 securities trading regulation [15]. Consider three representative rules: R_1 (Trading Mode), R_2 (Matching Priority), and R_3 (Execution Quantity), as shown in Fig. 1. Based



(a) Changed rules vs. affected test cases.

(b) Affected test cases identified by ESIM/RPTSP.

Fig. 2. Cascading impacts in a NYSE regulation and the limitations of cascading impact-unaware approaches.

on these rules, we can derive multiple system requirements. For example, from R_1 we obtain Req_1: *The trading system shall implement a continuous auction mechanism via a central limit order book, which must accept and process buy or sell orders submitted by market participants in real time.* These requirements are logically composed into a scenario $S : Req_1 \rightarrow Req_2 \rightarrow Req_3$, reflecting the execution lifecycle of a trade from order validation to order matching and execution determination. A corresponding set of test cases is organized along this scenario: C_1-C_2 exercise Req_1, C_3-C_8 exercise Req_2, and C_9-C_{10} exercise Req_3. These test cases form an execution chain, in which subsequent cases depend on the system states and outcomes established by the earlier ones.

Now consider changing the matching priority in R_2 from *time-then-price* to *price-then-time*, as shown in Fig. 1. Although R_1 and R_3 remain textually unchanged, this change causes cascading impacts across scenarios and test cases. Since R_3 depends on the matching results of R_2, the altered priority changes its matching sequence and execution context. Consequently, the impact extends beyond C_3-C_8 and invalidates the subsequent test cases C_9-C_{10}. For example, an order previously *Not Filled* may now be *Fully Filled*, requiring the subsequent test cases to be re-evaluated or regenerated.

This example demonstrates that regulatory changes can indirectly invalidate test cases of otherwise unchanged rules through *scenarios*, resulting in a significant *cascading impact*. A cascading impact occurs when modifications to a small subset of regulatory rules not only invalidate their directly associated test cases but also propagate through interdependent requirements within the scenarios, thereby affecting the test cases of other textually unchanged rules.

To assess the presence of cascading impacts in practice, we conducted an empirical study of regulatory updates from major exchanges such as NYSE [32] and NASDAQ [47], using historical test cases provided by our industrial partners. We take the NYSE 2025 securities trading regulation [14] as a representative example. Regulatory evolution is simulated by varying the ratio of changed rules from 0% to 100% and measuring the ratio of affected test cases, as shown in Fig. 2a.

The affected test ratio (green region) lies predominantly above the baseline $y = x$, revealing a significant amplification impact: changes to a small subset of rules typically invalidate a much larger fraction of test cases. When highly interdependent rules are modified first (upper bound), the cascading impact propagates rapidly, and the affected test ratio grows much faster than $y = x$, especially when fewer than 43% of rules are changed. Even when minimally dependent rules are changed first (lower bound), stepwise jumps above $y = x$ appear once more than 37% of rules are modified, indicating that individual rule changes can still trigger cascading impacts.

Together, the example and empirical results demonstrate that cascading impacts are mechanistically grounded and present in real-world regulatory evolution, highlighting the necessity of explicitly modeling and handling them in test case update and reuse to avoid incomplete updates and unnecessary maintenance effort during compliance evolution.

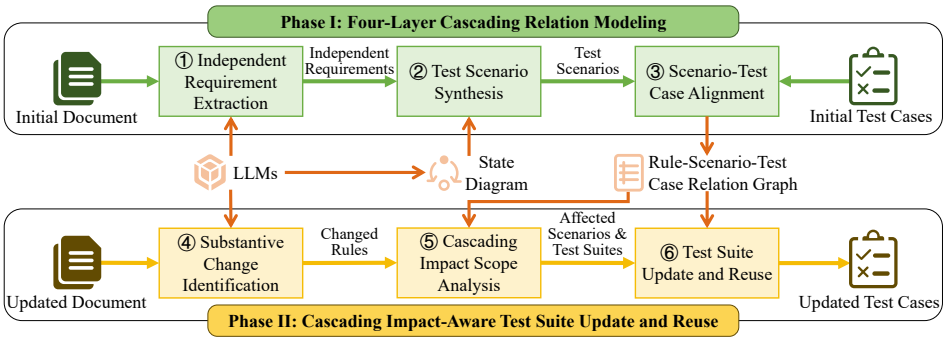


Fig. 3. The two-phase cascading impact-aware framework for test suite update and reuse.

3.2 Limitations of Existing Methods and Design Implications

We evaluate two SOTA test update and reuse methods, ESIM [4] and RPTSP [31], in terms of their ability to identify affected test cases under regulatory changes. Since both methods follow the same procedure to identify affected test cases and neither considers cascading impacts, they produce identical results, identifying only test cases directly linked to the changed rules, as shown in Fig. 2b.

Three distinct regions emerge. First, *under-identification* (green region) occurs when test cases indirectly invalidated by cascading impacts are missed, resulting in insufficient updates. Second, *erroneous reuse* (blue region) arises when test cases associated with unchanged rules are incorrectly reused, despite being semantically invalid due to corrupted execution contexts. Third, only test cases directly linked to changed rules are *correctly identified* (overlapping region). These results confirm that existing approaches are fundamentally cascading impact-unaware: they fail to trace how rule changes propagate through interdependent requirements and scenarios to test cases.

The observation directly motivates two key design implications. First, a test update and reuse framework must explicitly construct cascading relations among rules, requirements, scenarios, and test cases. Since multiple interdependent rules collectively define scenario-level execution logic, constructing scenarios is thus essential for accurately modeling rule interactions and cascading impacts across abstraction levels. Second, the framework must analyze change propagation through these cascading relations. Because changes to a small subset of rules may alter scenario semantics and invalidate multiple test cases, update and reuse decisions should be made at the scenario level.

Overall, these findings motivate the need for a cascading impact-aware approach that can systematically trace regulatory changes through scenarios and support precise, scenario-driven test case update and reuse under evolving regulations.

4 The Cascading Impact-Aware Approach for Test Suite Update and Reuse

4.1 Framework Overview

To address the frequent evolution of regulatory rules, this paper proposes CARE, a cascading impact-aware compliance testing evolution framework. Existing approaches often overlook the complex interdependencies among regulatory rules, leading to a dual maintenance dilemma of over-reuse and missed updates. To overcome this limitation, CARE is guided by a core design philosophy that transforms compliance test maintenance from an experience-driven, ambiguous process into a precise computational problem grounded in an explicit dependency network.

To this end, the framework adopts a hybrid *LLM + deterministic algorithm* paradigm, enabling precise maintenance through a modeling-first, analysis-second workflow. LLMs are used for semantically intensive tasks, such as structured requirement extraction, while deterministic algorithms

Initial Rules & Updates

Rule 1: For securities block trading, a "block" shall be at least 10,000 shares or a quantity of stock having a market value of $200,000 or more, whichever is less and whether selling or buying.

Rule 2 (Update): When facing market volatility, exchange systems will cancel buy (sell) orders that ~~are priced~~ above (below) the upper (lower) price band. ↳ are priced or could be traded

~~**Rule 3 (Delete):** Trading system will accept a maximum selling order up to 99 million shares.~~

Rule 4 (Add): A Non-DMM Market Maker will clear and settle its purchasing transactions through a registered clearing agency.

Initial Test Cases

	Market	Variety	Method	Operation	Quantity/Price	Event	Result	Next
Case 1	NYSE	Security	Block TD	Sell	10K SH, $200K	-	Succeed	Case 5, 6
Case 2	NYSE	Security	Block TD	Sell	20K SH, $300K	-	Succeed	Case 5, 6
Case 3	NYSE	Security	Block TD	Sell	5K SH, $200K	-	Fail	-
Case 4	NYSE	Security	Block TD	Sell	10K SH, $100K	-	Fail	-
Case 5	NYSE	Security	Block TD	Cancel Sell	Above Upper Bound	Market Volatility	Succeed	-
Case 6	NYSE	Security	Block TD	Cancel Sell	Below Upper Bound	Market Volatility	Fail	-
Case 7	NYSE	Security	Block TD	Buy	10K SH, $300K	-	Succeed	Case 10, 11
Case 8	NYSE	Security	Block TD	Buy	20K SH, $200K	-	Succeed	Case 10, 11
Case 9	NYSE	Security	Block TD	Buy	100M SH, $5K	-	Fail	-
Case 10	NYSE	Security	Block TD	Cancel Buy	Below Upper Bound	Market Volatility	Succeed	-
Case 11	NYSE	Security	Block TD	Cancel Buy	Above Lower Bound	Market Volatility	Fail	-

Fig. 4. Example of three initial rules (Rules 1-3), their updates, and part of the test cases for the initial rules.

construct the relation graph and perform change propagation analysis. By confining LLM usage to the early modeling stage, the framework limits error propagation caused by LLM hallucinations and improves the correctness and reliability of the final updated test cases.

The framework takes the initial regulatory documents, the updated regulatory documents, and the initial test cases as input, and outputs a set of test cases semantically aligned with the updated regulations. Its workflow consists of the following two core phases, as shown in Fig. 3:

Phase I: Four-Layer Cascading Relation Modeling. In this phase, an explicit dependency network is constructed to model the cascading relations among regulatory rules, requirements, scenarios, and test cases, enabling fine-grained traceability across abstraction levels. It comprises three steps. First, regulatory rules are decomposed into structured, independent requirements by extracting structured representations of regulatory constraints and consolidating related ones into self-contained requirements. Second, logically interdependent requirements are synthesized into scenarios that capture how multiple rules jointly govern execution behaviors. Finally, test cases are semantically aligned with the constructed scenarios, establishing traceable links from rules to test cases. Test cases associated with the same scenario are further aggregated into test suites, yielding a unified four-layer rule-requirement-scenario-test case cascading relation graph.

Phase II: Cascading Impact-Aware Test Suite Update and Reuse. Based on the constructed relation graph, this phase performs precise impact analysis and optimized maintenance of test assets. It likewise consists of three steps. First, substantive regulatory changes are identified by detecting semantic differences between the initial and updated requirements. Second, the cascading impacts of these changes are propagated through the rule-requirement-scenario-test case graph to accurately identify affected scenarios and test suites. Finally, unaffected portions of existing test suites are preserved and reused, while only the minimal set of impacted test suites is updated.

4.2 Phase I: Four-Layer Cascading Relation Modeling

Phase I aims to explicitly model the cascading relations among regulatory rules, requirements, scenarios, and test cases, laying the foundation for subsequent change impact analysis and test suite evolution. We illustrate this phase using three initial rules (*Rules 1-3*) derived from the NYSE Securities Trading Rules [12], their updated versions [14], and a subset of corresponding test cases, as shown in Fig. 4. The resulting four-layer cascading relation graph is depicted in Fig. 5.

4.2.1 Independent Requirement Extraction (Step ①). This step aims to disentangle coupled regulatory content and reduce the semantic gap between regulatory rules and test cases. To this end, we decompose complex rules into fine-grained, single-aspect requirements and organize them in a

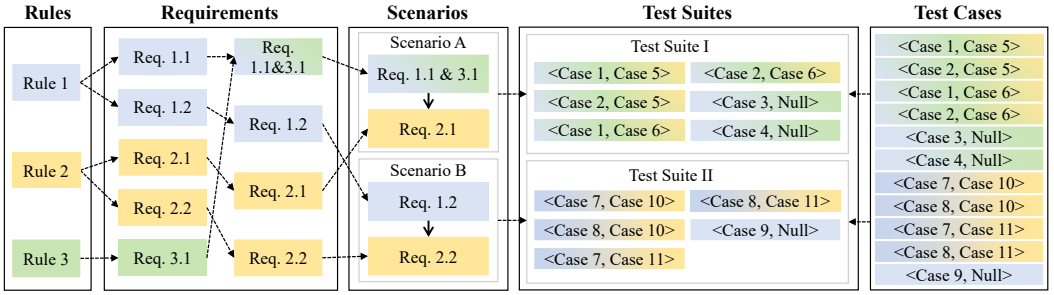


Fig. 5. The four-layer cascading relations for the initial rules (Rules 1-3) and test cases in Fig. 4. Gradient-colored boxes denote requirements or test cases obtained by merging multiple rules.

structured, test-aligned format. It thus consists of two sub-tasks: (i) extracting structured requirements to capture regulatory constraints, and (ii) consolidating related constraints into independent requirements that fully specify a single regulatory aspect. Here, all such requirements are functional requirements based on verifiability rather than the traditional "behavior vs. constraint" dichotomy, as they specify explicit constraints and can be operationalized into testable conditions.

Following Xue et al. [70], who propose a structured requirement representation for test case generation, we adopt it to formalize regulatory rules. Each requirement is represented in an *if-then* format, where *if* specifies triggering conditions or operations, and *then* defines the expected result, aligning with the precondition-assertion structure of test cases. Both clauses are formulated as logical combinations of *key-value* constraints connected by *and* or *or*. A set of core keys (e.g., *Actor*, *Operation*, and *Time*) is predefined, while additional keys can be dynamically introduced when needed, making the representation expressive and extensible across domains, while preserving sufficient semantic and structural information to facilitate downstream scenario synthesis, traceability alignment, and test suite evolution. Example 1 illustrates the structured requirements extracted from the three initial rules in Fig. 4. Each rule is decomposed into one or more requirements, where each requirement isolates a single constraint aspect: *Rule 1* is decomposed into *Req. 1.1* and *Req. 1.2*, *Rule 2* into *Req. 2.1* and *Req. 2.2*, and *Rule 3* into *Req. 3.1*.

Example 1: Structured Requirements for *Rules 1-3* in Fig. 4.

```
Req. 1.1
if Market is "NYSE" and Variety is "Security" and Method is "Block␣Trading"
   and Operation is "Sell"
then Quantity is "10,000␣shares␣or␣more" and Market Value is "$200,000␣or␣more"
Req. 1.2
if Market is "NYSE" and Variety is "Security" and Method is "Block␣Trading"
   and Operation is "Buy"
then Quantity is "10,000␣shares␣or␣more" and Market Value is "$200,000␣or␣more"
Req. 2.1
if Market is "NYSE" and Event is "Market␣Volatility" and Price is "Above␣Upper␣Bound"
then Operation is "Cancel␣Sell"
Req. 2.2
if Market is "NYSE" and Event is "Market␣Volatility" and Price is "Below␣Lower␣Bound"
then Operation is "Cancel␣Buy"
Req. 3.1
if Market is "NYSE" and Operation is "Sell"
then Quantity is "99␣million␣shares␣or␣less"
```

We employ SOTA LLMs (e.g., GPT-5.5 [54]) to extract *structured requirements* from regulatory rules via a unified prompt that integrates chain-of-thought (CoT) [66] and self-reflection [41]. Executing as a single inference call, the prompt guides the LLM through three logical stages:

(i) **Constraint Identification.** Given a regulatory rule, the LLM identifies relevant regulatory elements and assigns semantic labels, representing each as a *key-value* constraint. The stage outputs a set of atomic constraints, each capturing a specific rule condition.
(ii) **Requirement Construction.** The LLM organizes constraints into *if-then* requirements, placing condition-related constraints in the *if*-clause and outcome-related ones in the *then*-clause, with logical relations (*and*/*or*) explicitly defined. To ensure unambiguous triggering, disjunctive *if*-clauses are split into separate requirements, while the *then*-clauses remain unchanged. This stage outputs a set of structured requirements, each capturing a single regulatory aspect.
(iii) **Self-Reflection and Validation.** The LLM re-evaluates the requirements against the initial rule to ensure completeness and consistency by identifying missing constraints, resolving conflicts, and correcting semantic deviations, outputting the validated structured requirements.

In addition to the rule, the prompt also includes the full regulatory document, the predefined key schemas (*Actor*, *Time*, etc.), the formatting specifications for requirement representation, and three representative few-shot examples as contextual information. These signals help the LLM generate complete, consistent, and structured outputs while maintaining cross-domain extensibility.

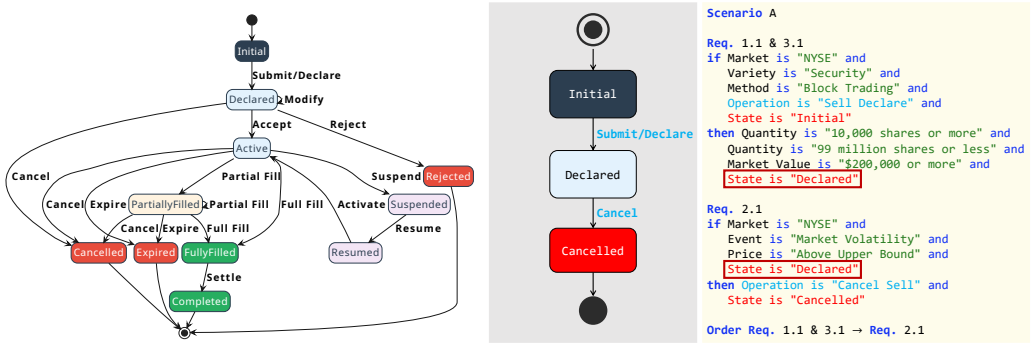
Next, structured requirements are consolidated into *independent requirements* via a deterministic merging algorithm. An independent requirement is a self-contained requirement that specifies all relevant constraints under a fixed set of triggering conditions. Specifically, structured requirements are first grouped by their *if*-clauses under two criteria: (i) requirements with different values on the same key are excluded; (ii) only those whose *if*-clauses exhibit a subset relation (the *key-value* constraints in one requirement's *if*-clause are contained in another's) are grouped for merging. Requirements with overlapping but non-subset *if*-clauses are not merged, as they constrain different condition dimensions. Within each group, the *then*-clauses are aggregated to form a unified independent requirement. For example, the *if*-clause of *Req. 3.1* is a subset of that of *Req. 1.1*, so their *then*-clause constraints are combined into *Req. 1.1 & 3.1*, as shown in Example 2.

Example 2: Independent Requirement for Req. 1.1 and Req. 3.1.

```
Req. 1.1 & 3.1
if Market is "NYSE" and Variety is "Security" and Method is "Block␣Trading"
   and Operation is "Sell"
then Quantity is "10,000␣shares␣or␣more" and Quantity is "99␣million␣shares␣or␣less"
   and Market Value is "$200,000␣or␣more"
```

4.2.2 Scenario Synthesis (Step ②). In this step, we synthesize scenarios by uncovering dependencies among independent requirements through an algorithmic process. We consider two types of inter-requirement relations: *explicit temporal relations* and *implicit data dependencies*. Explicit temporal relations are identified by detecting keywords in the "Time" field of independent requirements, including "before", "after", and "until". These keywords induce a partial ordering that directly indicates which requirement should precede or follow another in a scenario. Implicit data dependencies are captured by analyzing system state transitions triggered by requirement operations. We employ an LLM to construct a comprehensive state diagram through two sequential prompting steps:

(i) **State Definition.** The input consists of all operations extracted from independent requirements, together with two representative example rules per operation. The LLM is prompted to infer the corresponding *pre-state* and *post-state* for each operation, producing a table that maps operations to state transitions. During this process, semantically equivalent or highly similar state transitions are normalized into unified state representations, reducing state fragmentation and improving the consistency of subsequent dependency discovery and scenario synthesis.



(a) Constructed financial state diagram. (b) Scenario synthesis based on the state diagram.

Fig. 6. State-diagram-based scenario synthesis. Fig. 6a shows the financial state diagram constructed from requirement operations and inferred state transitions, while Fig. 6b shows how independent requirements are chained into executable scenarios through temporal relations and state dependencies.

(ii) **State Diagram Construction.** Using the state-transition table, the LLM connects states through operations to construct a financial state diagram, represented as a PlantUML diagram [62] in which nodes denote system states and directed edges correspond to operation-triggered transitions. Figure 6a shows the state diagram constructed for the financial domain.

During scenario synthesis, each requirement's pre- and post-states are inferred from the state diagram based on its operation and added to the *if*- and *then*-clauses, respectively. A data dependency is established when the post-state of one requirement is identical to the pre-state of another. Finally, requirements linked by temporal relations or data dependencies are chained to form scenarios, each representing a coherent sequence of regulatory constraints that make inter-rule dependencies and cascading effects explicit. Fig. 6b shows the scenario synthesized from *Req. 1.1 & 3.1* and *Req. 2.1*. Executing *Req. 1.1 & 3.1* transitions the system state from *Initial* to *Declared* after a block sell order is declared, which enables the subsequent applicability of *Req. 2.1* for order cancellation under market volatility. Linking these two independent requirements via their inferred state transitions yields *Scenario A*, representing a realistic execution flow in compliance testing.

4.2.3 Scenario-Test Case Alignment (Step ③). To bridge the semantic gap between synthesized scenarios and concrete test cases, we perform scenario-test case alignment using a semantic-aware matching algorithm, as shown in Algorithm 1. The objective is to determine whether a test case chain matches a scenario by first aligning independent requirements with test cases and then validating their structural consistency at the scenario level.

At the *requirement-test case* level, each test case is matched to candidate requirements based on the correspondence between key-value constraints and execution outcomes (Lines 3-14). For *numerical constraints* (e.g., *Time*, *Quantity*, and *Price*), test case instances are compared against the value ranges of the same attribute within a requirement while jointly considering the execution result: a match is established when an in-range instance results in a successful outcome or an out-of-range instance results in a failure, and all other cases are treated as non-matching (Lines 7-9). For *non-numerical constraints*, alignment is determined by exact or semantic equivalence (Lines 10-12): identical values are directly matched, while non-identical ones are embedded using SentenceTransformer [57] and compared via cosine similarity [38]. Two values are considered aligned only if their similarity exceeds a threshold of $\tau = 0.9$, which is selected through extensive experimental evaluation by systematically varying τ to balance false alignments and missed matches.

Algorithm 1: Semantic-Aware Scenario-Test Case Alignment

Input : T: set of test case chains; S: set of identified scenarios; τ: cosine similarity threshold.
Output: R: mapping from each scenario to aligned test case chains.

1 $R \leftarrow Map()$; // Mapping from scenarios $s \in S$ to test case chains $tc \in T$
2 **foreach** *scenarios and test case chains* $(s, tc) \in S \times T$ **do**
3 $C \leftarrow Map()$; // Mapping from requirements $r \in s$ to test cases $t \in tc$
4 **foreach** *requirements and test cases* $(r_i, t_j) \in s \times tc$ **do**
5 $cover_r \leftarrow \emptyset$; // Keys of r_i covered by t_j
6 **foreach** *constraints* $(key_i^a, value_i^a) \in r_i$ *and test data* $(attr_j^b, inst_j^b) \in t_j$ **do**
7 **if** *IsNumConstraint*(key_i^a) *and IsNumConstraint*$(attr_j^b)$ *and* $key_i^a = attr_j^b$ **then**
8 **if** *NumSatisfyRange*$(value_i^a, inst_j^b) \oplus$ *IsResultFail*(t_j) **then**
9 $cover_r \leftarrow cover_r \cup \{key_i^a\}$; // Numeric range and expected outcome match
10 **else if** *!IsNumConstraint*(key_i^a) *and !IsNumConstraint*$(attr_j^b)$ **then**
11 **if** $value_i^a = inst_j^b$ *or CosineSimilarity*$(value_i^a, inst_j^b) \geq \tau$ **then**
12 $cover_r \leftarrow cover_r \cup \{key_i^a\}$; // Non-Numeric values match
13 **if** $cover_r = r_i.keys()$ **then**
14 $C[r_i] \leftarrow t_j$; // t_j matches r_i
15 **if** *IsOneToOne(C, s, tc)* ***or*** *(IsPrefix(C, s, tc)* ***and*** *IsResultFail(tc))* **then**
16 $R[s] \leftarrow R[s] \cup \{tc\}$; // tc aligns with s
17 **return** R

At the *scenario-test case chain* level, a test case chain is considered aligned with a scenario if (i) a one-to-one correspondence exists between requirements and test cases while preserving execution order, or (ii) the test case chain forms a valid prefix of the scenario and terminates with a failure outcome, indicating partial execution caused by regulatory violation (Lines 15-17). By aligning test cases with synthesized scenarios, this step aggregates test cases of the same scenario into *test suites*, as shown in Fig. 5. Each test suite contains test cases that collectively cover all requirements within a scenario and explicitly reflect the execution flow induced by interdependent rules.

For example, *Cases 1-4* are aligned with *Req. 1.1 & 3.1*, and *Cases 5-6* with *Req. 2.1*; their resulting test case chain is further aligned with *Scenario A*. These test cases are therefore aggregated into *Test Suite I*, while test cases aligned with other scenarios are grouped analogously.

4.3 Phase II: Cascading Impact-Aware Test Suite Update and Reuse

Phase II leverages the rule-requirement-scenario-test case cascading relations modeled in Phase I to perform cascading impact-aware test suite update and reuse for the updated regulatory document, thereby ensuring precise identification of affected test cases and efficient maintenance.

4.3.1 Substantive Change Identification (Step ④). To accurately identify substantive regulatory changes, we perform change identification at the level of independent requirements rather than directly comparing raw text, enabling us to distinguish semantic changes that affect compliance behavior from superficial textual edits that do not. We first reuse the method in *Step* ① to generate independent requirements for the updated document. Each updated requirement is then compared against those of the initial document using the semantic-aware matching strategy of *Step* ③. Two requirements are considered identical if either (i) all key-value constraints in their if-then structures match exactly, or (ii) their semantic similarity exceeds the threshold $\tau = 0.9$. Any addition, deletion, or modification of a requirement, even in a single attribute, is treated as a substantive change.

This requirement-level comparison enables the framework to effectively filter out non-impactful edits. For example, although *Rule 2* in Fig. 4 undergoes textual revisions, its independent requirements before and after revision remain identical and therefore introduce no substantive change.

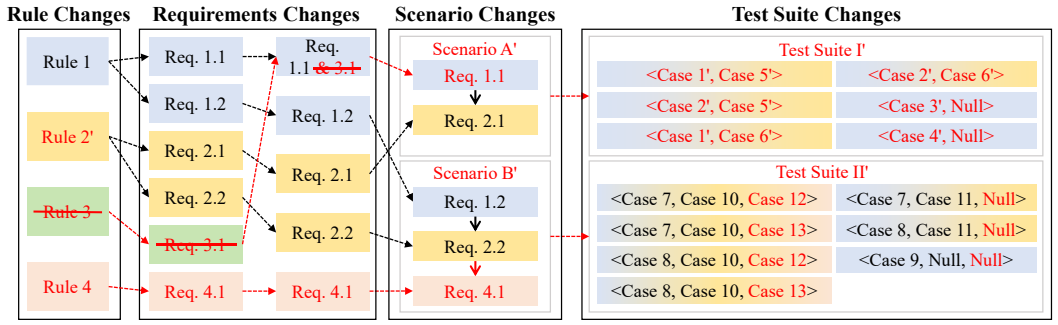


Fig. 7. Substantive change identification and cascading impact scope analysis for the rule updates in Fig. 4. Elements highlighted in red indicate modified rules, affected requirements, scenarios, and test cases.

In contrast, deleting *Rule 3* removes its corresponding requirement *Req. 3.1*, while adding *Rule 4* introduces *Req. 4.1*. Both changes are identified as substantive updates, as shown in Fig. 7.

4.3.2 Cascading Impact Scope Analysis (Step ⑤). This step analyzes how substantive regulatory changes propagate through scenarios to test suites, leveraging the cascading relations modeled in Phase I. For each changed rule, we trace how its affected requirements participate in scenarios and test suites, and determine the reuse decision by the changed requirement's position in the scenario. If no scenario is affected, all test suites can be *fully reused*. If the change occurs after some requirements in a scenario, *partial reuse* is enabled: test cases for unaffected prefix requirements are reused, and only the impacted suffix is regenerated. If the change occurs at the head of a scenario, the test suite becomes invalid, resulting in *no reuse* and requiring full regeneration.

As shown in Fig. 7, changing *Rule 2* to *Rule 2'* does not alter its requirements and therefore leaves all scenarios and test suites reusable. In contrast, adding *Rule 4* introduces a new requirement *Req. 4.1*, which is appended after *Req. 2.2* due to a data dependency, updating *Scenario B* to *Scenario B'*. In this case, test cases for *Req. 1.2* and *Req. 2.2* can be reused, while new test cases (*Cases 12-13*) should be generated for *Req. 4.1*. Finally, deleting *Rule 3* modifies the head of *Scenario A* by replacing *Req. 1.1 & 3.1* with *Req. 1.1*, rendering *Test Suite I* invalid and requiring full regeneration.

4.3.3 Test Suite Update and Reuse (Step ⑥). In the final step, test suites are updated or reused according to the analyzed reuse level. For *full reuse*, existing test suites are preserved without modification. For *partial reuse*, only test cases associated with affected or newly introduced requirements are regenerated, while reusable test cases are retained and integrated into the updated test suites. For *no reuse*, all test suites of the impacted requirements and scenarios are regenerated. This strategy enables precise test evolution while avoiding unnecessary regeneration.

New test suites are generated through an algorithmic process that uses *key-value* pairs from requirements as constraints. The overall procedure consists of three steps:

(i) **Test Data Generation.** We apply classical test generation techniques, including *equivalence partitioning* and *boundary value analysis* [28], to produce candidate data for each constraint. For numerical constraints, representative valid and invalid values are selected from the range, together with boundary conditions around the limits. For non-numerical constraints with a finite domain, all values are enumerated; otherwise, a valid (X) and a generalized invalid value (not X) are generated. This ensures coverage of positive, negative, and boundary behaviors.

(ii) **Test Case Generation.** Test data are combined into test cases according to the logical relations among constraints, including *and*, *or*, and *if-then*. To avoid a combinatorial explosion, we apply *pairwise coverage* and *representative sampling* [37, 45]. Pairwise coverage includes every

Algorithm 2: Cascading Impact-Aware Test Case Update and Reuse

Input : $Rule_{ini}$: initial rules; $Rule_{upd}$: updated rules; TC: initial test cases ; G: rule-requirement-scenario-test case cascading relation graph between $Rule_{ini}$ and TC.
Output: TC': updated test cases.

1 $R_{ini} \leftarrow GetRequirements(G, Rule_{ini})$; // Initial independent requirements
2 $R_{upd} \leftarrow GenerateRequirements(Rule_{upd})$; // Updated independent requirements
3 $R_{chg} \leftarrow IdentifyChanges(R_{ini}, R_{upd})$; // Substantive requirement changes
4 $S_{upd} \leftarrow \emptyset$; // Updated scenarios with change positions
5 **foreach** *changed requirements* $r \in R_{chg}$ **do**
6 $s', pos, TC_{inv} \leftarrow AnalyzeImpactScope(G, r)$; // Cascading impact analysis
7 $S_{upd} \leftarrow S_{upd} \cup \{(s', pos)\}$;
8 $TC \leftarrow TC \setminus TC_{inv}$; // Remove invalid test cases
9 $TC' \leftarrow TC$; // Preserve reusable test cases
10 **foreach** *updated scenarios and change positions* $(s', pos) \in S_{upd}$ **do**
11 $TC_{upd} \leftarrow GenerateTests(s', pos)$; // Generate updated test cases
12 $TC' \leftarrow TC' \cup TC_{new}$;
13 **return** TC' ;

combination of positive and negative values for each *pair* of constraints rather than enumerating all combinations, while representative sampling selects only a typical valid/invalid value and a boundary value for each input, efficiently covering key interactions without redundancy. A test case is labeled *invalid* if any constraint is violated, and *valid* otherwise.

(iii) **Test Suite Assembly.** The generated test cases are grouped according to their corresponding scenarios to form updated test suites, preserving logical and functional context while integrating new or modified cases for cascading impact-aware updates.

This procedure produces valid, invalid, and boundary test cases while respecting both individual constraints and their interactions, yielding test suites that are compact, effective, and ready for practical evaluation. Example 3 illustrates the updated test suites corresponding to Fig. 7.

Example 3: Updated Test Suites in Fig. 7.

	Market	Variety	Method	Operation	Quantity/Price	Event	Result	Next
Case 1'	NYSE	Security	Block TD	Sell Declare	50K SH, $500K	-	Succeed	**Case 5',6'**
Case 2'	NYSE	Security	Block TD	Sell Declare	10K SH, $200K	-	Succeed	**Case 5',6'**
Case 3'	NYSE	Security	Block TD	Sell Declare	5K SH, $500K	-	Fail	-
Case 4'	NYSE	Security	Block TD	Sell Declare	50K SH, $100K	-	Fail	-
Case 5'	NYSE	Security	Block TD	Cancel Sell	Above Upper Band	Market Volatility	Succeed	-
Case 6'	NYSE	Security	Block TD	Cancel Sell	Below Upper Band	Market Volatility	Fail	-
Case 12	NYSE	Security	-	Clear Buy	-	-	Succeed	-
Case 13	NYSE	Security	-	Settle Buy	-	-	Succeed	-

To summarize the overall workflow of Phase II, Algorithm 2 formalizes the cascading impact-aware update and reuse process. Given updated rules, the algorithm identifies substantial changes (Lines 1-3), propagates their impact across scenarios and test suites via cascading relations (Lines 4-8), and finally updates and reuses test suites according to the identified reuse levels (Lines 9-13).

5 Evaluation

To evaluate the performance of CARE, we conducted a comprehensive experimental study to answer the following research questions:

(1) **RQ1:** How effective and efficient is CARE in updating and reusing test cases and test suites under regulatory rule changes compared to experts, LLMs, and SOTA update and reuse methods?

Table 1. Characteristics of the six constructed evaluation datasets. #X means the number of X. The last three columns show the ratio of changed rules, scenarios, and test cases, illustrating the cascading impact.

Dataset	Initial Document	#Rule	#Scenario	#Test Case	Updated Document	#Updated Rule	#Affected Scenario	#Affected Case
1	New York Stock Exchange Auction Market Bids and Offers Rules (2015) [11]	24	406	1587	New York Stock Exchange Auction Market Bids and Offers Rules (2017) [13]	12 (50.0%)	319 (78.6%)	1313 (82.7%)
2	New York Stock Exchange Equities Trading Rules (2016) [12]	10	92	338	New York Stock Exchange Equities Trading Rules (2019) [14]	8 (80.0%)	86 (93.5%)	320 (94.7%)
3	Nasdaq Stock Market Options Trading Rules (2021) [42]	28	103	130	Nasdaq Stock Market Options Trading Rules (2025) [44]	8 (28.6%)	81 (78.6%)	112 (86.2%)
4	Shanghai Stock Exchange Trading Rules (2nd Ed., 2020) [17]	34	386	648	Shanghai Stock Exchange Trading Rules (2023) [19]	15 (44.1%)	236 (61.1%)	420 (64.8%)
5	Shenzhen Stock Exchange Trading Rules (2021.3) [18]	27	224	470	Shenzhen Stock Exchange Trading Rules (2023) [20]	9 (33.3%)	106 (47.3%)	254 (54.0%)
6	Tokyo Stock Exchange Bond Trading Regulations (2015) [21]	12	104	562	Tokyo Stock Exchange Bond Trading Regulations (2025) [22]	3 (25.0%)	44 (42.3%)	414 (73.7%)

(2) **RQ2:** How does each component of CARE contribute to the update and reuse of test cases and suites, and to what extent does explicitly handling cascading impacts affect the final results?
(3) **RQ3:** To what extent do LLM hallucinations and error propagation affect the effectiveness of CARE, and how effectively does our hybrid LLM-algorithm design mitigate these effects?
(4) **RQ4:** How well does CARE generalize across multiple highly regulated domains?

Implementation and Experimental Setup. We implement CARE as a modular Python-based pipeline for automated test suite update and reuse under regulatory evolution. As shown in Figure 3, the pipeline consists of six sequential steps, with all intermediate artifacts explicitly preserved to facilitate traceability, reproducibility, and optional human inspection. Among these, Steps ①, ②, and ④, which involve structured requirement extraction and state diagram construction, are achieved by invoking advanced LLM APIs. To ensure robust and reproducible generation, we adopt deterministic decoding by setting the temperature to 0 and disabling stochastic token sampling. To further mitigate the non-determinism inherent in LLM outputs, all LLM-based methods were executed three times, and the reported metrics are averages across these repetitions. All experiments are conducted on a standard machine without requiring specialized computational resources.

5.1 Experiment I: Effectiveness and Efficiency of Test Suite Update and Reuse

Datasets. As no public benchmarks exist for evaluating test case update and reuse under regulatory evolution, we constructed six real-world datasets from five major exchanges to evaluate CARE in the financial domain. Each dataset includes an initial regulatory document, its updated version, and the corresponding test cases before and after the update. The documents are authentic financial rules sourced from official exchange websites and reflect actual regulatory revisions, while the test cases are anonymized industrial cases from a production stock trading system, provided by our industry partners, ensuring both realism and practical relevance. Importantly, the six datasets exhibit varying degrees of regulatory change, covering diverse ratios of updated rules, affected scenarios, and test cases, which enables a systematic evaluation of cascading impacts from rules to scenarios and test cases. Table 1 summarizes their key characteristics. Note that the document updates span 2-10 years to ensure that more changed rules and affected test cases are included, although regulations change even more frequently in practice, as described in Section 1.
Competitors. We compare CARE against four representative categories of baselines: (i) *Domain Experts*, including three senior testing engineers with 3-5 years of financial experience. They are industrial collaborators from the product and architecture teams of a securities company, working on compliance testing. Following industry-standard workflows, they manually evolve test suites by linking rules to test cases and updating them accordingly, without introducing intermediate

requirements or scenarios. (ii) *End-to-End (E2E) LLMs*, represented by GPT-5.5 [54], Grok-4.3 [67], and Gemini-3.1 [7]. These are SOTA LLMs from different providers, widely used in prior work and representative of high-performing systems for complex reasoning. (iii) *SOTA Requirements-Based Test Update and Reuse Methods*, including ESIM [4] and RPTSP [31]. (iv) *SOTA Test Generation Approach*, represented by LLM4Fin [70], which uses BERT [8] to regenerate test cases from scratch and serves as an efficiency-only baseline. Since the implementations of ESIM and RPTSP are not publicly available, we reimplement them by strictly following the procedures described in the original papers. LLM4Fin is excluded from effectiveness comparisons, as it does not leverage historical test cases; nevertheless, we report its effectiveness metrics for reference, since the review time in efficiency evaluation depends on the correctness of test cases.

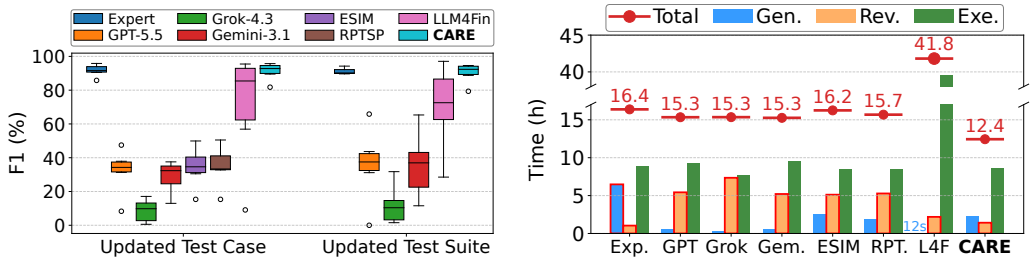
Metrics. Due to strict security policies at financial institutions, neither source code nor code-level metrics are available. Following prior work [24, 63], we assess test quality by comparing the generated test cases against real-world industrial cases, which serve as the ground truth. The system under test is accessed only through exposed testing interfaces, enabling test execution and time measurement without revealing implementation details. We evaluate effectiveness at two levels: *test case* and *test suite*. A *test case* corresponds to a requirement and is correct if all key-value pairs are identical to the ground truth. A *test suite* corresponds to a scenario and is correct only if all its constituent test cases are correct. Effectiveness is measured using *Precision*, *Recall*, and *F1* across three categories: *regenerated* (newly generated for updated rules), *reused* (unchanged from existing tests), and *updated* (the union of reused and regenerated tests). To assess scenario-level coverage, we adopt *Business Scenario Coverage (BSC)* [70], defined as: $BSC = \frac{p_t}{p_a}$, where p_t is the number of covered scenarios and p_a is the total. Higher BSC values indicate broader coverage of regulatory requirements and scenarios under rule changes. To evaluate efficiency, we measure three types of time costs: (i) *Generation Time*, including test reuse (when applicable) and new test generation; (ii) *Review Time*, expert effort required to inspect and correct tests until full correctness is achieved; and (iii) *Execution Time*, the time required to execute the finalized tests. We also report the *Total Time*, which sums the three to reflect the overall efficiency of the maintenance process.

Experimental Procedure. Each of the six datasets was provided to the three domain experts, who manually updated the test cases. For E2E LLMs, we provided the initial and updated documents together with the initial test cases, and adopted a structured three-step CoT prompting process: (i) *Rule-Test Case Association Construction*: the LLM maps rules to test cases while considering inter-rule dependencies, producing a rule-test case relation table; (ii) *Change and Impact Analysis*: using the initial and updated documents and the relation table, the LLM identifies rule changes and affected test cases; (iii) *Test Case Update*: the LLM updates affected test cases according to the revised rules and outputs the final test suite. For ESIM and RPTSP, we followed their default settings. For LLM4Fin, only the updated documents were provided as input to generate the corresponding test cases. To evaluate the impact of different LLM choices within our framework, we ran CARE using GPT-5.5, Grok-4.3, and Gemini-3.1, and reported results separately. After generation, all outputs were reviewed by experts and then executed through the testing interface. We recorded the time spent and used a script to automatically compute all metrics. Since ESIM and RPTSP do not account for inter-rule dependencies, they do not provide test suite-level metrics, and LLM4Fin generates all test cases from scratch and thus does not report regenerated or reused metrics.

Experimental Results. Table 2 reports the effectiveness results. Domain experts achieve the best overall performance. CARE attains average updated test case and suite F1 of 91.2% and 90.3% across the six datasets, closely matching expert performance and outperforming SOTA reuse-based methods by up to 164%, while balancing precision and recall for both regenerated and reused tests. The slightly higher precision than recall in reuse indicates a conservative update strategy,

Table 2. Comparison of Precision (Pre., %), Recall (Rec., %), F1 (%), and Business Scenario Coverage (BSC, %) for Regenerated (Reg.), Reused (Reu.), and Updated (Upd.) Test Cases and Test Suites (if applicable) produced by Domain Experts, three E2E LLMs, SOTA test reuse tools ESIM and RPTSP, SOTA test generation tool LLM4Fin, and CARE on the six financial evaluation datasets. Reported values are averages over the six datasets. "-" indicates that the corresponding results cannot be obtained due to methodological constraints.

Methods	Reg. Test Case			Reu. Test Case			Upd. Test Case				Reg. Test Suite			Reu. Test Suite			Upd. Test Suite		
	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1	BSC	Pre.	Rec.	F1	Pre.	Rec.	F1	Pre.	Rec.	F1
Experts	**93.9**	89.0	**91.3**	91.0	**93.4**	91.2	**94.0**	89.6	91.7	87.3	**89.2**	**90.9**	**90.0**	85.4	**93.4**	87.8	**90.1**	92.3	**91.2**
GPT-5.5	27.2	13.1	15.9	23.4	73.6	34.2	41.1	29.7	33.8	69.3	32.4	12.8	18.9	27.1	74.5	36.5	48.3	31.9	37.6
Grok-4.3	15.1	0.2	3.5	46.2	14.9	15.8	38.7	4.2	9.9	46.8	16.3	0.6	4.2	41.1	13.9	14.1	37.4	6.3	12.9
Gemini-3.1	28.5	12.3	15.6	31.2	68.9	37.6	42.8	23.9	30.4	65.1	37.4	11.6	18.5	32.1	69.2	39.4	54.1	31.3	37.2
ESIM	27.0	12.1	15.5	23.0	91.0	35.6	42.0	32.1	34.5	67.0		-			-			-	
RPTSP	24.3	11.9	15.2	23.8	92.8	36.9	40.6	31.9	34.8	72.3		-			-			-	
LLM4Fin		-			-		75.2	78.0	70.9	73.9		-			-		73.7	71.2	70.2
CARE (GPT-5.5)	91.3	**89.1**	89.4	**98.0**	92.1	**94.8**	93.5	92.2	**92.4**	**87.6**	87.8	89.9	89.0	**96.2**	91.1	**93.2**	88.1	**92.9**	90.5
CARE (Grok-4.3)	87.4	88.6	88.0	96.1	88.3	92.0	90.3	90.8	90.6	84.7	84.6	87.5	86.0	94.7	86.9	90.6	85.7	88.8	87.2
CARE (Gemini-3.1)	88.6	**89.1**	88.9	96.8	89.2	92.8	91.1	**92.4**	91.8	85.9	85.2	88.9	87.0	95.8	88.0	91.7	86.9	90.1	88.5



(a) F1 Distribution. (b) Time Consumption.

Fig. 8. Comparison of the F1 Distribution of Updated Test Cases and Test Suites metrics across the six financial datasets (left), and the Generation (Gen.), Review (Rev.), and Execution (Exe.) Time Consumption for each competitor (right), with time costs associated with human effort highlighted in red borders.

where some potentially reusable cases are regenerated to ensure completeness. In addition, CARE achieves the highest BSC, indicating a strong ability to capture and cover regulatory scenarios under rule evolution. In contrast, all E2E LLMs perform poorly, with F1 around 30% or lower. This is primarily because they fail to accurately establish correspondence between rules and test cases, leading to incorrect reuse and insufficient updates of invalid test cases. ESIM and RPTSP exhibit performance comparable to E2E LLMs. Although they achieve higher recall for reuse, their low precision demonstrates an inability to capture cascading impacts, as they rely solely on direct rule-test mappings and ignore indirect dependencies. LLM4Fin explicitly considers inter-rule dependencies during test generation, enabling it to achieve higher F1 than ESIM and RPTSP. However, its performance remains inferior to CARE since it does not reuse historical test cases.

We further analyze the impact of different LLMs within CARE. As shown in Table 2, CARE consistently achieves strong performance across GPT-5.5, Grok-4.3, and Gemini-3.1, indicating that the framework is robust to the choice of LLM and that its effectiveness mainly stems from the cascading impact-aware design rather than a specific LLM. Among them, GPT-5.5 performs best in most cases; we therefore use **GPT-5.5** as the default LLM in subsequent experiments.

Figure 8a illustrates the distribution of updated test case and test suite F1 across the six datasets, revealing the stability of each competitor. Domain experts exhibit the lowest variance. CARE ranks second, with only slightly higher variance due to its partial reliance on LLMs, yet it substantially

Table 3. Module configuration of CARE, its ablated variants, and related competitors used in the ablation study. ✓ indicates that the corresponding module is enabled. Some ablated variants align with existing test reuse or generation approaches and are highlighted with the same background colors.

Category	Method	Requirement Extraction ($\mathcal{R}$)	Scenario Synthesis ($\mathcal{S}$)	Scenario-Test Alignment ($\mathcal{G}$)	Change Identification (C)	Impact Analysis ($\mathcal{I}$)	Test Update & Reuse ($\mathcal{T}$)
Baseline	CARE	✓	✓	✓	✓	✓	✓
Variants (No Cascading Awareness)	w/o $\mathcal{I}$	✓	✓	✓	✓		✓
	w/o $\mathcal{S}$ + $\mathcal{I}$	✓		✓	✓		✓
	w/o $\mathcal{R}$ + $\mathcal{S}$ + $\mathcal{I}$			✓		✓	✓
	w/o $\mathcal{G}$ + C + $\mathcal{T}$	✓	✓				
Competitors (No Cascading Awareness)	RPTSP	✓		✓	✓		✓
	ESIM			✓		✓	✓
	LLM4Fin	✓	✓				

outperforms all other methods by mitigating stochasticity through LLM self-reflection and deterministic algorithms, as further evaluated in Experiment III. In contrast, E2E LLMs, ESIM, and RPTSP exhibit higher variance, reflecting their susceptibility to LLM stochasticity and hallucinations. LLM4Fin is the least stable, as it regenerates all test cases without leveraging historical artifacts.

Figure 8b reports the efficiency results. All reuse-based methods incur comparable total times (15.3-16.4 hours), with CARE being the fastest at 12.4 hours. In contrast, LLM4Fin requires substantially more time (41.8 hours) because it must regenerate and execute all test cases from scratch. While E2E LLMs minimize generation time, their low precision and recall lead to extensive expert review. ESIM and RPTSP incur moderate generation time but still require substantial review effort due to incorrect updates and reuse. In contrast, CARE produces more accurate results, significantly reducing review time and minimizing manual effort. Regarding execution time, reuse-based methods incur similar costs as they execute a comparable number of regenerated test cases, while LLM4Fin executes all tests without reuse and thus takes the longest. Overall, CARE reduces both expert workload and redundant execution, achieving superior efficiency under regulatory evolution.

Conclusion: CARE achieves expert-level effectiveness in test suite update and reuse while significantly reducing human review effort and improving testing efficiency under regulatory evolution.

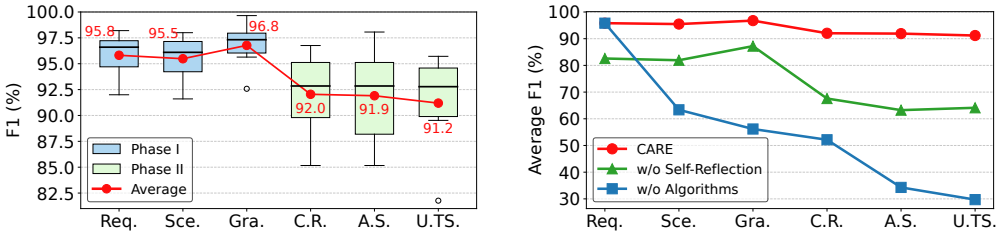
5.2 Experiment II: Contribution of Components and Cascading Impact Awareness

Experimental Design. To quantify the contribution of each component in CARE and assess the importance of cascading impact awareness, we conduct an ablation study with the configurations summarized in Table 3. Due to strong interdependencies among modules (e.g., C depends on the rule-test case relation graph constructed by $\mathcal{G}$ and cannot operate independently), we design a sequence of ablated variants that progressively remove cascading-awareness capabilities: (i) *w/o* $\mathcal{I}$, which disables impact analysis to isolate the impact of cascading propagation; (ii) *w/o* $\mathcal{S} + \mathcal{I}$ (aligned with RPTSP), which further removes scenario synthesis and thus ignores inter-rule dependencies; (iii) *w/o* $\mathcal{R} + \mathcal{S} + \mathcal{I}$ (aligned with ESIM), which additionally eliminates structured requirement extraction and relies solely on natural language similarity; and (iv) *w/o* $\mathcal{G} + C + \mathcal{T}$ (aligned with LLM4Fin), which abandons test reuse entirely and regenerates test cases from scratch. This progressive strategy enables a controlled investigation of how cascading awareness and each component contribute to effective test update and reuse, yielding variants that resemble existing approaches. The datasets, metrics, and experimental procedures are identical to those in Experiment I.

Experimental Results. The results in Table 4 highlight the critical role of cascading impact awareness. When the impact analysis module $\mathcal{I}$ is removed, the updated test case F1 drops sharply from 91.2% to 58.8%. As cascading-related components ($\mathcal{S}$ and $\mathcal{R}$) are further disabled, performance deteriorates dramatically to 33.3% and 28.6%, respectively. A similar trend is observed for regenerated and reused test cases and suites, demonstrating that neglecting cascading impacts leads to severe

Table 4. Comparison of CARE with the four ablated variants and three related competitors on the average F1 of regenerated, reused, and updated test cases and test suites across the six financial evaluation datasets. Δ denotes the relative performance drop of each variant and related competitor compared to CARE.

Method	Reg. Test Case		Reu. Test Case		Upd. Test Case		Reg. Test Suite		Reu. Test Suite		Upd. Test Suite	
	F1	Δ	F1	Δ	F1	Δ	F1	Δ	F1	Δ	F1	Δ
CARE	88.1	-	93.9	-	91.2	-	88.4	-	93.1	-	90.3	-
w/o $\mathcal{I}$	57.8	34.4 ↓	59.5	36.6 ↓	58.8	35.5 ↓	59.7	32.5 ↓	69.1	25.8 ↓	61.9	31.5 ↓
w/o $\mathcal{S}+\mathcal{I}$	14.9	83.1 ↓	37.9	59.7 ↓	33.3	63.5 ↓		-		-		-
w/o $\mathcal{R}+\mathcal{S}+\mathcal{I}$	13.8	84.4 ↓	35.8	61.9 ↓	28.6	68.6 ↓		-		-		-
w/o $\mathcal{G}+\mathcal{C}+\mathcal{T}$		-		-	79.7	12.6 ↓		-		-	79.6	11.8 ↓
RPTSP	15.2	82.8 ↓	36.9	60.7 ↓	34.8	61.8 ↓		-		-		-
ESIM	15.5	82.5 ↓	35.6	62.1 ↓	34.5	62.2 ↓		-		-		-
LLM4Fin		-		-	70.9	22.2 ↓		-		-	70.2	22.2 ↓



(a) F1 distribution and mean.

(b) Comparison on average F1.

Fig. 9. F1 of intermediate artifacts generated by CARE, including generated requirements (Req.), synthesized scenarios (Sce.), relation graph (Gra.), changed rules (C.R.), affected scenarios (A.S.), and updated test suites (U.TS.), compared with variants without LLM self-reflection and an LLM-only pipeline.

error propagation during test update and reuse. Comparable approaches that lack cascading impact awareness, including RPTSP and ESIM, exhibit similar degradation. These results confirm that explicitly modeling and handling cascading impacts is essential for effective test update and reuse under regulatory evolution. In contrast, the *w/o* $\mathcal{G}+\mathcal{C}+\mathcal{T}$ variant and LLM4Fin exhibit a more moderate performance drop (11.8%-22.2%). By abandoning test case alignment and reuse, these settings effectively adopt a regeneration-from-scratch strategy, which discards valuable contextual knowledge embedded in existing test suites, resulting in lower updated F1 than CARE.

Conclusion: While all components of CARE contribute to test suite update and reuse, explicit cascading impact modeling and handling are the decisive factors underlying its effectiveness.

5.3 Experiment III: Impact of Hallucinations and Error Propagation

Experimental Design. This experiment examines how LLM hallucinations arise and propagate across the stages of CARE, and how our self-reflection mechanisms and deterministic algorithms mitigate their impact. We first evaluate the correctness of each intermediate artifact of CARE on the six datasets in Experiment I, including generated requirements, synthesized scenarios, relation graphs, changed rules, affected scenarios, and updated test suites. Then, we compare it with two variants: (i) CARE *w/o Self-Reflection*, which removes LLM self-reflection while keeping the rest unchanged; and (ii) CARE *w/o Algorithms*, where all intermediate artifacts are generated by LLMs without deterministic algorithms. For each configuration, ground truth is constructed through expert review, and F1 is reported at every step.

Experimental Results. As shown in Figure 9a, CARE achieves consistently high F1 across all intermediate steps. Hallucinations mainly occur in LLM-dependent modules, including requirement

Table 5. Characteristics of Automotive (Datasets 7-9) and Power (Datasets 10-11) evaluation datasets.

Dataset	Initial Document	#Rule	#Scenario	#Test Case	Updated Document	#Updated Rule	#Affected Scenario	#Affected Case
7	The 2021 New Jersey Driver Manual [5]	73	216	422	The 2025 New Jersey Driver Manual [6]	19 (26.4%)	86 (39.8%)	205 (48.6%)
8	Road Traffic Safety Law of PRC (2011) [52]	124	173	878	Road Traffic Safety Law of PRC (2021) [53]	37 (29.8%)	96 (55.5%)	562 (64.0%)
9	Japanese Road Traffic Act (2017) [50]	35	71	290	Japanese Road Traffic Act (2026) [51]	18 (51.4%)	45 (63.4%)	177 (61.0%)
10	EN 50438:2013 [25]	112	150	375	EN 50549-1:2019 [26]	94 (83.9%)	145 (96.7%)	369 (98.4%)
11	GB/T 19964-2012 [48]	32	55	134	GB/T 19964-2024 [49]	13 (40.6%)	32 (58.2%)	79 (59.0%)

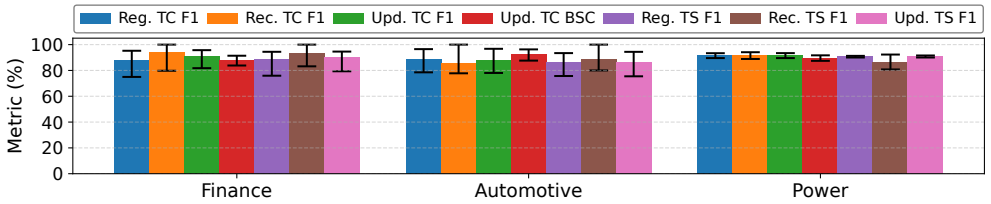


Fig. 10. Generalize performance of CARE on the average F1 and BSC of Regenerated, Reused, and Updated Test Cases (TC) and Test Suites (TS) across the Finance, Automation, and Power domains.

extraction and change identification, but are effectively mitigated by self-reflection, yielding high F1 of 95.8% and 92.0%. Subsequent modules rely on deterministic algorithms and thus do not introduce new hallucinations, resulting in stable performance with only mild degradation. However, compared to Phase I, Phase II exhibits higher variance, as it not only introduces new hallucinations but also compounds errors from Phase I. Notably, a local rebound from scenario to relation graph indicates that our similarity-based algorithm can filter noise and partially correct earlier errors.

Figure 9b further validates our design choices. Removing LLM self-reflection results in a noticeable drop in requirement extraction and change identification, amplifying subsequent errors. The LLM-only pipeline exhibits the most severe error propagation: although its early-stage performance is comparable, errors accumulate rapidly in later stages, particularly in relation graph construction and affected scenario identification, resulting in a sharp decline in final test case correctness. In contrast, CARE suppresses hallucinations via self-reflection and constrains error propagation via deterministic algorithms, maintaining high F1 across intermediate artifacts and final outputs.

Conclusion: CARE achieves high performance across all intermediate steps, where LLM hallucinations are mitigated by self-reflection, and initial errors are reduced by deterministic algorithms.

5.4 Experiment IV: Generalizability Across Diverse Regulatory Domains

Experimental Design. To evaluate the cross-domain generalizability of CARE, we extend our study beyond finance to two additional domains: Automotive and Power. Following Experiment I, we construct three datasets for autonomous driving systems and two for power control systems, as summarized in Table 5. This experiment adopts a self-comparison setting to examine whether CARE can consistently handle regulatory evolution under diverse regulatory frameworks and environments. When migrating to other domains, CARE requires only minimal effort, as its prompts and algorithms are designed to be domain-agnostic. It only needs light adaptation to domain-specific representations, such as attribute keys or rule expression formats. All metrics and experimental procedures strictly follow those in Experiment I.

Experimental Results. As shown in Figure 10, CARE maintains consistently high performance across finance, automotive, and power domains. The F1 of updated test cases and test suites remains

above 86% in all domains (finance: 91.2% / 90.3%; automotive: 88.2% / 86.1%; power: 91.5% / 90.8%). The consistently small gaps and low variance between updated and reused metrics demonstrate that CARE effectively models and handles cascading impacts across heterogeneous regulatory frameworks and domain-specific environments, enabling accurate test updates, effective test reuse, and stable performance under diverse regulatory domain evolutions.
Conclusion: CARE demonstrates strong cross-domain generalizability across diverse regulatory domains, supporting effective and stable test suite update and reuse under regulatory evolution.

6 Threats to Validity

Dependence on structured requirement representations. Our approach relies on the assumption that regulatory rules can be accurately captured using structured requirement representations. While this abstraction enables explicit cascading relation construction and deterministic reasoning, it also bounds the applicability of CARE. If the chosen representation fails to express certain regulatory semantics such as implicit constraints or highly contextual exceptions, the subsequent scenario synthesis and impact analysis may become ineffective. We encountered a small number of such cases in our experiments, which can now be handled manually. Improving the expressiveness and adaptability of requirement representations remains an important direction for future work.
Residual LLM hallucinations and human dependence. Although CARE substantially mitigates the impact of LLM hallucinations through self-reflection and deterministic algorithms, error propagation persists. In practice, selectively validating intermediate artifacts rather than final test cases can significantly reduce human effort, as these artifacts are more compact and easier to inspect. Nevertheless, the framework still relies on human oversight to ensure correctness in safety-critical settings, and fully autonomous compliance testing remains an open challenge.
Dependence on Commercial LLMs. Our experiments rely on commercial LLMs accessed via APIs, which poses a threat to reproducibility, as model providers may update models without version disclosure and prior versions are not accessible. To mitigate this, CARE is designed to be model-agnostic, and we additionally provide a fine-tuned open-weight model for structured requirement generation to support more reproducible experimentation.

7 Conclusion

This paper addresses the challenge of cascading impact in compliance test maintenance by proposing CARE, a cascading impact-aware test evolution framework. Built on a four-layer Rule-Requirement-Scenario-Test Case traceability map, CARE explicitly models and accurately propagates regulatory changes, mitigating both test over-reuse and insufficient update. Extensive experiments across multiple compliance domains show that CARE achieves expert-level performance (F1 > 86%), significantly outperforming existing approaches while reducing maintenance effort. These results demonstrate the importance of explicit cascading impact modeling for compliance test maintenance. Future work will focus on large-scale industrial deployment and human-machine collaboration to further improve the reliability of change propagation. Overall, CARE provides a practical and scalable solution for maintaining compliance test suites in evolving regulatory environments.

8 Data Availability

Our code, prompts, and evaluation datasets are available at https://github.com/13luoyu/CARE.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant Nos. 92582203 and 62372176. We would like to express our sincere gratitude to Prof. Nan Niu of the University of North Florida for his invaluable insights and guidance for this work.

References

[1] Suriya Priya R Asaithambi and Stan Jarzabek. 2013. Towards test case reuse: A study of redundancies in android platform test libraries. In *International Conference on Software Reuse*. Springer, 49–64. doi:10.1007/978-3-642-38977-1_4

[2] Suriya Priya R Asaithambi and Stan Jarzabek. 2015. Pragmatic Approach to Test Case Reuse-A Case Study in Android OS BiDiTests Library. In *International Conference on Software Reuse*. Springer, 122–138. doi:10.1007/978-3-319-14130-5_9

[3] Lizhi Cai, Weiqin Tong, Zhenyu Liu, and Juan Zhang. 2009. Test case reuse based on ontology. In *2009 15th IEEE Pacific Rim International Symposium on Dependable Computing*. IEEE, 103–108. doi:10.1109/PRDC.2009.25

[4] Xiaomeng Chen, Jinbo Wang, Shan Zhou, Panpan Xue, and Jiao Jia. 2021. Test Case Reuse based on ESIM Model. In *2021 8th International Conference on Dependable Systems and Their Applications*. IEEE, 700–705. doi:10.1109/DSA52907.2021.00101

[5] New Jersey Motor Vehicle Commission. 2021. The 2021 New Jersey Driver Manual. https://static.epermittest.com/media/filer_public/4e/21/4e2120cd-6a57-4d0c-b97b-23db47de85f6/new-jersey-drivers-manual-2022.pdf. Accessed: 2026-07-22.

[6] New Jersey Motor Vehicle Commission. 2025. The 2025 New Jersey Driver Manual. https://www.nj.gov/mvc/pdf/license/drivermanual.pdf. Accessed: 2026-07-22.

[7] Google DeepMind. 2026. Gemini 3. "https://deepmind.google/models/gemini/". Accessed: 2026-07-22.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186. doi:10.18653/v1/N19-1423

[9] Nicolas Devos, Christophe Ponsard, Jean-Christophe Deprez, Renaud Bauvin, Benedicte Moriau, and Guy Anckaerts. 2012. Efficient reuse of domain-specific test knowledge: An industrial case in the smart card domain. In *2012 34th International Conference on Software Engineering*. IEEE, 1123–1132. doi:10.1109/ICSE.2012.6227107

[10] André Di Thommazo, Kamilla Camargo, Elis Montoro Hernandes, Gislaine Gonçalves, Jefferson Pedro, Anderson Belgamo, and Sandra CPF Fabbri. 2015. Using the Dependence Level Among Requirements to Priorize the Regression Testing Set and Characterize the Complexity of Requirements Change.. In *Proceedings of the 17th International Conference on Enterprise Information Systems*, Vol. 1. 231–241. doi:10.5220/0005468902310241

[11] New York Stock Exchange. 2015. New York Stock Exchange Auction Market Bids and Offers Rules (2015). "https://history.srorules.com/app/#/2025-08-23/tax/Dealings-and-Settlements+Dealings-and-Settlements-Rules-45-299C-+Auction-Market-Bids-and-Offers-Rules-63-80B-/document/Rule-64.-Bonds-Rights-and-100-Share-Unit-Stocks.html?uniqueId=0WiJe5P4EshEbN5I_3sADw==&corpus=nyse". Accessed: 2026-07-22.

[12] New York Stock Exchange. 2016. New York Stock Exchange Equities Trading Rules (2016). "https://history.srorules.com/app/#/2025-08-23/tax/Rules+Rules+Rule-7P-EQUITIES-TRADING/document/Rule-7P-EQUITIES-TRADING.html?uniqueId=zHrE1fKB9Dk4x_Fl4pvfhg==&corpus=nyse". Accessed: 2026-07-22.

[13] New York Stock Exchange. 2017. New York Stock Exchange Auction Market Bids and Offers Rules (2017). "https://nyseguide.srorules.com/rules/09013e2c8553e79b". Accessed: 2026-07-22.

[14] New York Stock Exchange. 2019. New York Stock Exchange Equities Trading Rules (2019). "https://nyseguide.srorules.com/rules/adabe2b07d5e100088ed005056881d2301". Accessed: 2026-07-22.

[15] New York Stock Exchange. 2025. New York Stock Exchange Rules. "https://nyseguide.srorules.com/rules". Accessed: 2026-07-22.

[16] New York Stock Exchange. 2025. NYSE Rules: What's New. "https://nyseguide.srorules.com". Accessed: 2026-07-22.

[17] Shanghai Stock Exchange. 2020. Shanghai Stock Exchange Trading Rules (Second Revision in 2020). "https://www.sse.com.cn/lawandrules/sselawsrules/repeal/rules/c/10118619/files/b39b8db2daa74eac916d8b3d6907af51.docx". Accessed: 2026-07-22.

[18] Shenzhen Stock Exchange. 2021. Shenzhen Stock Exchange Trading Rules (Revised in March 2021). "https://docs.static.szse.cn/www/lawrules/rule/trade/current/W020210331716400647184.pdf". Accessed: 2026-07-22.

[19] Shanghai Stock Exchange. 2023. Shanghai Stock Exchange Trading Rules (Revised in 2023). "https://www.sse.com.cn/lawandrules/sselawsrules2025/repeal/rules/c/10824490/files/dcbe58edb194451d93f19b1f7dd8fb4c.docx". Accessed: 2026-07-22.

[20] Shenzhen Stock Exchange. 2023. Shenzhen Stock Exchange Trading Rules (Revised in 2023). "https://docs.static.szse.cn/www/lawrules/rule/repeal/rules/W020230217564423808793.pdf". Accessed: 2026-07-22.

[21] Tokyo Stock Exchange. 2015. Tokyo Stock Exchange Business Regulations (2015). "https://www.jpx.co.jp/rules-participants/rules/revise/00-archives-10.html". Accessed: 2026-07-22.

[22] Tokyo Stock Exchange. 2025. Tokyo Stock Exchange Business Regulations (2025). "https://resource.lexis-asone.jp/jpx/rule/tosho_regu_201305070003001.html". Accessed: 2026-07-22.

[23] Tokyo Stock Exchange. 2025. Tokyo Stock Exchange Rule Amendments Comparison Table. "https://www.jpx.co.jp/rules-participants/rules/revise/index.html". Accessed: 2026-07-22.

[24] Sakina Fatima, Taher A Ghaleb, and Lionel Briand. 2023. Flakify: A black-box, language model-based predictor for flaky tests. *IEEE Transactions on Software Engineering* 49, 4 (2023), 1912–1927. doi:10.1109/TSE.2022.3201209

[25] European Committee for Electrotechnical Standardization. 2013. EN 50438:2013 Requirements for micro-generating plants to be connected in parallel with public low-voltage distribution networks. https://standards.iteh.ai/catalog/standards/clc/ec27ccfe-33fa-4434-bc21-72a2182fcbf3/en-50438-2013. Accessed: 2026-07-22.

[26] European Committee for Electrotechnical Standardization. 2019. EN 50549-1:2019 Requirements for generating plants to be connected in parallel with distribution networks - Part 1: Connection to a LV distribution network - Generating plants up to and including Type B. https://standards.iteh.ai/catalog/standards/clc/948bd277-da91-40c7-b7e0-bbf561035638/en-50549-1-2019. Accessed: 2026-07-22.

[27] Monica Giffin, Olivier de Weck, Gergana Bounova, Rene Keller, Claudia Eckert, and P. John Clarkson. 2009. Change Propagation Analysis in Complex Technical Systems. *Journal of Mechanical Design* 131, 8 (2009), 081001. doi:10.1115/1.3149847

[28] Dorothy Graham, Erik van Veenendaal, Isabel Evans, and Rex Black. 2006. *Foundations of software testing: ISTQB certification*. Cengage Learning.

[29] Siqi Gu, Zhibiao Liu, Qiaoqing Liang, Zhimin Deng, and Xiangyu Xiao. 2023. Test Case Reuse Method Based on Deep Semantic Matching. In *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 593–599. doi:10.1109/QRS-C60940.2023.00022

[30] Hui Guan, Hang Xu, and Lie Cai. 2024. Requirement dependency extraction based on improved stacking ensemble machine learning. *Mathematics* 12, 9 (2024), 1272. doi:10.3390/math12091272

[31] Irum Ilays, Yaser Hafeez, Nabil Almashfi, Sadia Ali, Mamoona Humayun, Muhammad Aqib, and Ghadah Alwakid. 2024. Towards improving the quality of requirement and testing process in agile software development: an empirical study. *Computers, Materials, & Continua* 80, 3 (2024), 3761. doi:10.32604/cmc.2024.053830

[32] Inc. Intercontinental Exchange. 2025. The New York Stock Exchange. "https://www.nyse.com". Accessed: 2026-07-22.

[33] Pilsu Jung, Seonah Lee, and Uicheon Lee. 2024. Automated code-based test case reuse for software product line testing. *Information and Software Technology* 166 (2024), 107372. doi:10.1016/j.infsof.2023.107372

[34] Sungwon Kang, Haeun Baek, Jungmin Kim, and Jihyun Lee. 2015. Systematic software product line test case derivation for test data reuse. In *2015 IEEE 39th Annual Computer Software and Applications Conference*, Vol. 3. IEEE, 433–440. doi:10.1109/COMPSAC.2015.174

[35] Martin Kellogg, Martin Schäf, Serdar Tasiran, and Michael D Ernst. 2020. Continuous compliance. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 511–523. doi:10.1145/3324884.3416593

[36] Oleksandr Kosenkov, Parisa Elahidoost, Tony Gorschek, Jannik Fischbach, Daniel Mendez, Michael Unterkalmsteiner, Davide Fucci, and Rahul Mohanani. 2025. Systematic mapping study on requirements engineering for regulatory compliance of software systems. *Information and Software Technology* 178 (2025), 107622. doi:10.1016/j.infsof.2024.107622

[37] D Richard Kuhn, Dolores R Wallace, and Albert M Gallo. 2004. Software fault interactions and implications for software testing. *IEEE transactions on software engineering* 30, 6 (2004), 418–421. doi:10.1109/TSE.2004.24

[38] Alfirna Rizqi Lahitani, Adhistya Erna Permanasari, and Noor Akhmad Setiawan. 2016. Cosine similarity to determine similarity measure: Study case in online essay assessment. In *2016 4th International conference on cyber and IT service management*. IEEE, 1–6. doi:10.1109/CITSM.2016.7577578

[39] Tobias Landsberg, Christian Dietrich, and Daniel Lohmann. 2022. TASTING: Reuse Test-case Execution by Global AST Hashing.. In *International Conference on Software Technologies*. 33–45. doi:10.5220/0011139200003266

[40] Wenwu Li and Miyi Duan. 2014. A study on the software test case reuse model of feature oriented. In *2014 IEEE 3rd International Conference on Cloud Computing and Intelligence Systems*. IEEE, 241–246. doi:10.1109/CCIS.2014.7175736

[41] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594. https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf

[42] Nasdaq Stock Market. 2021. Nasdaq Stock Market Options Trading Rules (2021). "https://listingcenter.nasdaq.com/RulebookVersionHistory.aspx?docId=8522&version=16". Accessed: 2026-07-22.

[43] Nasdaq Stock Market. 2025. Nasdaq Listing Center: Rulebook and Rule Filings. "https://listingcenter.nasdaq.com/rulebook/nasdaq/rulefilings". Accessed: 2026-07-22.

[44] Nasdaq Stock Market. 2025. Nasdaq Stock Market Options Trading Rules (2025). "https://listingcenter.nasdaq.com/RuleBook/Nasdaq/rules/Nasdaq%20Options%203". Accessed: 2026-07-22.

[45] Glenford J Myers, Tom Badgett, Todd M Thomas, and Corey Sandler. 2004. *The art of software testing* (2 ed.). John Wiley & Sons. https://web.tecgraf.puc-rio.br/~raquelg/LIVRO_TheArtOfSoftwareTesting.pdf

[46] Zhang Nan, Chai Haiyan, and Han Xinyu. 2018. Research on the reuse of test case for warship equipment software. In *2018 IEEE International Conference on Software Quality, Reliability and Security Companion*. IEEE, 64–69. doi:10.1109/QRS-

C.2018.00025

[47] Inc. Nasdaq. 2025. NASDAQ: Stock Market, Data Updates, Reports & News. "https://www.nasdaq.com". Accessed: 2026-07-22.

[48] National Standardization Management Committee of China. 2012. GB/T 19964-2012 Technical requirements for connecting photovoltaic power station to power system. https://ebook.chinabuilding.com.cn/zbooklib/bookpdf/probation?SiteID=1&bookID=105517. Accessed: 2026-07-22.

[49] National Standardization Management Committee of China. 2024. GB/T 19964-2024 Technical requirements for connecting photovoltaic power station to power system. https://openstd.samr.gov.cn/bzgk/std/newGbInfo?hcno=40D8691DFD7EC3CBA423CCBA65D262F3. Accessed: 2026-07-22.

[50] National Diet of Japan and Ministry of Justice. 2017. Japanese Road Traffic Act. https://laws.e-gov.go.jp/law/335AC0000000105/20170312_427AC0000000040. Accessed: 2026-07-22.

[51] National Diet of Japan and Ministry of Justice. 2026. Japanese Road Traffic Act. https://laws.e-gov.go.jp/law/335AC0000000105/20260614_506AC0000000059. Accessed: 2026-07-22.

[52] Standing Committee of the National People's Congress of China. 2011. Road Traffic Safety Law of the People's Republic of China. https://flk.npc.gov.cn/detail?id=2c909fdd678bf17901678bf7201d05ff. Accessed: 2026-07-22.

[53] Standing Committee of the National People's Congress of China. 2021. Road Traffic Safety Law of the People's Republic of China. https://flk.npc.gov.cn/detail?id=ff8081817ab231eb017abd617ef70519. Accessed: 2026-07-22.

[54] OpenAI. 2026. ChatGPT. "https://chatgpt.com/". Accessed: 2026-07-22.

[55] Kamalendu Pal. 2020. Framework for Reusable Test Case Generation in Software Systems Testing. In *Software Engineering for Agile Application Development*. IGI Global, 212–229. doi:10.4018/978-1-7998-2531-9.ch009

[56] Sachin Patel and Ramesh Kumar Kollana. 2014. Test Case Reuse in Enterprise Software Implementation–An Experience Report. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, 99–102. doi:10.1109/ICST.2014.22

[57] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics. doi:10.18653/v1/D19-1410

[58] Ya-Qing Shi, Song Huang, and Jin-Yong Wan. 2022. A Reuse-oriented Clustering Method for Test Cases. In *2022 9th International Conference on Dependable Systems and Their Applications*. IEEE, 153–162. doi:10.1109/DSA56465.2022.00028

[59] Luis Alvaro de Lima Silva, Lori RF Machado, and Leonardo Emmendorfer. 2024. A Case and Cluster-Based Framework for Reuse and Prioritization in Software Testing. In *Proceedings of the 20th Brazilian Symposium on Information Systems*. 1–10. doi:10.1145/3658271.3658312

[60] Baradwaj Bandi Sudakara. 2025. Leading Cross-Functional QA in Healthcare: A Playbook for Automation and Compliance at Scale. *Journal of Computer Science and Technology Studies* 7, 8 (2025), 937–945. https://al-kindipublisher.com/index.php/jcsts/article/view/10660

[61] Hamid Tabani, Leonidas Kosmidis, Jaume Abella, Francisco J Cazorla, and Guillem Bernat. 2019. Assessing the adherence of an industrial autonomous driving framework to iso 26262 software guidelines. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6. doi:10.1145/3316781.3317779

[62] PlantUML Team. 2026. PlantUML. https://plantuml.com. Accessed: 2026-07-22.

[63] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020). doi:10.48550/arXiv.2009.05617

[64] Bangchao Wang, Rong Peng, Yuanbang Li, Han Lai, and Zhuo Wang. 2018. Requirements traceability technologies and technology transfer decision support: A systematic review. *Journal of Systems and Software* 146 (2018), 59–79. doi:10.1016/j.jss.2018.09.001

[65] Wentao Wang, Faryn Dumont, Nan Niu, and Glen Horton. 2022. Detecting software security vulnerabilities via requirements dependency analysis. *IEEE Transactions on Software Engineering* 48, 5 (2022), 1665–1675. doi:10.1109/TSE.2020.3030745

[66] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837. https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[67] xAI. 2026. Grok. "https://grok.com/". Accessed: 2026-07-22.

[68] Xinyue Xu, Sinong Chen, Zhonghao Guo, and Xiangxian Chen. 2025. PSTR: A Test Case Reuse Method Based on Path Similarity. *IEEE Access* 13 (2025), 3175–3187. doi:10.1109/ACCESS.2024.3524596

[69] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. 2024. Domain Knowledge is All You Need: A Field Deployment of LLM-Powered Test Case Generation in FinTech Domain. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. ACM, 314–315. doi:10.1145/3639478.3643087

[70] Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. 2024. LLM4Fin: Fully Automating LLM-Powered Test Case Generation for FinTech Software Acceptance Testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1643–1655. doi:10.1145/3650212.3680388

[71] Wansheng Yang, Fei Deng, Siyou Ma, Linbo Wu, Zhe Sun, and Chi Hu. 2021. Test case reuse based on software testing knowledge graph and collaborative filtering recommendation algorithm. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion*. IEEE, 67–76. doi:10.1109/QRS-C55045.2021.00020

[72] He Zhang, Juan Li, Liming Zhu, Ross Jeffery, Yan Liu, Qing Wang, and Mingshu Li. 2014. Investigating dependencies in software requirements for change propagation analysis. *Information and Software Technology* 56, 1 (2014), 40–53. doi:10.1016/j.infsof.2013.07.001

[73] Shunran Zhang, Zhengmin Jiang, Ming Sang, Yunjie Han, Huiyun Li, et al. 2025. A Comprehensive Survey on Driving Compliance Assessment Methodologies for Autonomous Vehicles. *IEEE Intelligent Transportation Systems Magazine* (2025). doi:10.1109/MITS.2025.3548869

Received 2026-01-30; accepted 2026-06-25